Project Proposal

Project Title: Smart Expense & Habit Tracker

Team name: "Byte_Builders"

Team members-
1.Tushar Sarker
ID: 231-115-171
2. Rasheda Chowdhury Sanju
ID: 231-115-187

Batch:58th
Section:E

Course name: Web programming lab & Project 300

Submitted To
Abu Jafar Md.Jakaria
Senior Lecturer, department of C.S.E.
Metropolitan University, Sylhet

Date of submission: 08-03-2026

1. Overview

Smart Expense and Habit Tracker is a web-based application developed using the MERN stack. The system is designed to help users manage their daily expenses and monitor personal habits such as study time, sleep duration, and physical exercise. The main problem this project addresses is the lack of structured personal financial awareness and habit monitoring among students and young professionals.

Many people record expenses manually in notebooks or simple note applications. These methods do not provide organized reports, pattern analysis, or intelligent alerts. In addition, habit tracking is usually done separately using different applications, which makes it difficult to understand the relationship between financial behavior and daily routines. This project aims to solve that problem by combining expense tracking and habit analysis into a single integrated platform.

The target users of this system are students, freelancers, and young professionals who want to improve financial discipline and maintain healthy daily habits. The system will allow users to track expenses, monitor habits, analyze monthly behavior, and receive smart alerts based on spending patterns.

This project is important because it promotes financial awareness and personal discipline through structured data analysis. It also provides practical experience in building a full-stack web application using modern technologies such as React, Next.js, Node.js, Express, and MongoDB.

2. Project Details

2.1 Type of System

Smart Expense is a web-based personal management system. It can be classified as a behavior tracking and financial management system. The application combines two major components: expense tracking and habit analysis. It is developed as a Single Page Application (SPA) using the MERN stack.

The system allows users to manage their personal financial records and daily habits within a centralized dashboard. It focuses on structured data storage, analytical reporting, and intelligent alert generation.

2.2 Scope of the Project

The scope of this project defines what will be included in the system and what will not be included.

Included in the Project

- User registration and secure login system
- Personal dashboard for each user
- Expense tracking module (Create, Read, Update, and Delete operations)
- Habit tracking module (daily input for study, sleep, and exercise)
- Weekly and monthly summary reports
- Data visualization using charts and graphs
- Rule-based smart alert system
- Profile management
- Responsive web interface
- Deployment on a cloud hosting platform

Not Included in the Project

- Mobile native application (Android or iOS)
- Integration with banking APIs
- Real-time AI or machine learning prediction engine
- Online payment system
- Multi-language support

2.3 Main Modules or Components of the System

The system will consist of the following main modules:

1. Authentication Module

This module will handle user registration, login, logout, and secure session management using JWT-based authentication.

2. Expense Management Module

This module will allow users to add daily expenses with categories such as food, transport, study, and entertainment. Users will be able to edit and delete records. All expense data will be stored in MongoDB.

3. Habit Tracking Module

This module will allow users to record daily habit data, including study hours, sleep duration, and exercise time. The system will store this information for analytical purposes.

4. Analytics and Dashboard Module

This module will generate visual reports such as monthly expense summaries, category-wise charts, and habit consistency graphs. It will help users understand their behavior patterns.

5. Smart Alert Module

This module will apply rule-based logic to detect unusual spending or declining habit performance. The system will generate alerts based on predefined conditions, such as excessive spending in a category.

6. Profile and Settings Module

This module will allow users to update personal information and manage account settings.

3. Project Features

The features of Expense AI are divided into functional features and system-level features. These features describe what the system will do and how users will interact with it.

3.1 Core Features

1. **User Authentication System**
   Users will be able to create an account and log in securely. Passwords will be encrypted, and authentication will be managed using JSON Web Tokens (JWT).

2. **Expense Entry and Management**
   Users will be able to add daily expenses with the following details:
   - Date
   - Amount
   - Category
   - Optional description

   Users can update or delete any previously added expense record.

3. **Expense Categorization**

   Expenses will be grouped into categories such as

   - Food
   - Transportation
   - Education
   - Shopping
   - Entertainment
   - Others

4. **Habit Tracking System**

   Users will be able to record daily habits, including

   - Study hours
   - Sleep duration
   - Exercise time

The system will store daily entries and calculate consistency over time.

5. Dashboard with Data Visualization
The dashboard will provide:

- Monthly expense summary
- Category-wise charts
- Weekly and monthly habit graphs
- Overall financial overview

Charts will help users understand spending patterns and habit trends.

6. Smart Alerts and Notifications

The system will generate alerts when:

- Spending exceeds a defined monthly limit
- A specific category shows abnormal increase
- Habit performance decreases over time

These alerts will be rule-based and automated.

7. Profile Management
Users will be able to update their profile information and manage account settings.

8. Responsive Design
The application will be fully responsive and usable on desktop, tablet, and mobile browsers.

3.2 Additional Recommended Features

To strengthen the project quality, the following recommended features will also be included:

- Clean and structured user interface
- Proper form validation
- Error handling and meaningful system messages
- Secure backend routing
- Organized folder structure
- RESTful API design
- Deployment on a cloud platform
- Version control using Git

4. Functional and Non-Functional Requirements

4.1 Functional Requirements

Functional requirements describe what the system must do.

1. **User Registration and Login**

    - The system shall allow new users to register using name, email, and password.
    - The system shall allow registered users to log in securely.
    - The system shall validate user credentials before granting access.
    - The system shall allow users to log out safely.

2. **Expense Management**

    - The system shall allow users to add daily expense records.
    - The system shall allow users to edit existing expense records.
    - The system shall allow users to delete expense records.
    - The system shall store expense data in the database.
    - The system shall categorize expenses automatically.

3. **Habit Tracking**

    - The system shall allow users to record daily habit information.
    - The system shall store habit data for each date.
    - The system shall calculate weekly and monthly summaries.
    - The system shall display habit consistency reports.

4. **Dashboard and Reports**

    - The system shall generate monthly expense summaries.
    - The system shall display graphical charts for category-wise expenses.
    - The system shall show trend analysis for habits.
    - The system shall update reports dynamically when data changes.

5. **Smart Alert System**

    - The system shall detect excessive spending.
    - The system shall generate alerts based on predefined rules.
    - The system shall notify users through dashboard messages.

6. **Profile Management**

    - The system shall allow users to update profile information.
    - The system shall allow users to change their password.

4.2 Non-Functional Requirements

Non-functional requirements describe how the system should perform.

1. **Performance**
    - The system shall load within 3 seconds under normal conditions.
    - API responses shall be efficient and optimized.
2. **Security**
    - User passwords shall be encrypted using secure hashing.
    - The system shall use JWT-based authentication.
    - The system shall prevent unauthorized access to user data.
3. **Usability**
    - The system shall provide a clean and simple user interface.
    - The system shall be easy to navigate.
    - Forms shall include validation and clear error messages.
4. **Reliability**
    - The system shall maintain data consistency.
    - The system shall handle errors properly without crashing.
5. **Scalability**
    - The system architecture shall support future expansion.
    - The database structure shall allow adding new features easily.
6. **Compatibility**
    - The system shall work on modern web browsers such as Chrome, Edge, and Firefox.
    - The system shall be responsive for different screen sizes.

5. Technology Stack and Tools

This project will be developed using the MERN stack architecture. The MERN stack is a modern full-stack JavaScript framework that allows both frontend and backend development using JavaScript.

5.1 Frontend Technologies

The frontend of the system will be responsible for user interface and user interaction.

- React.js (v19.2)
  React will be used to build reusable UI components and manage application state efficiently.
- Next.js (v16.1.6)
  Next.js will be used for routing, server-side rendering support, performance optimization, and overall application structure.

The frontend will be developed without using separate HTML and CSS files. Styling will be handled using modern component-based styling approaches supported within the React and Next.js ecosystem.

5.2 Backend Technologies

The backend will handle business logic, authentication, database operations, and API development.

- Node.js (v25.8.0)
  Node.js will be used as the runtime environment for executing server-side JavaScript.
- Express.js (v5.2.1)
  Express will be used to build RESTful APIs and manage routing.

5.3 Database Technologies

- MongoDB (v8.2): MongoDB will be used as the primary NoSQL database to store user information, expense records, and habit tracking data.
- MongoDB Atlas MongoDB Atlas will be used as the cloud database hosting service for storing and managing the project database online.
- MongoDB Compass will be used as the graphical interface tool to visualize, manage, and monitor the MongoDB database during development.

5.4 Development Tools

The following tools will be used during development:

- Visual Studio Code – Code editor
- Git – Version control system
- GitHub – Code repository management
- Postman – API testing
- MongoDB Compass – Database visualization and management
- Vercel – Frontend deployment
- Render or similar cloud platform – Backend deployment

5.5 Version Control and Deployment Strategy

The project will use Git for version control. All source code will be managed through GitHub repositories. Separate branches will be maintained for development and production.

The frontend will be deployed using Vercel, and the backend will be deployed on a cloud hosting service. The database will be hosted using MongoDB Atlas.

This technology selection ensures scalability, maintainability, and modern development standards.

6. Project Timeline

The project will be completed within a period of three months. The development process will be divided into structured phases to ensure systematic progress and timely completion.

6.1 Development Plan Overview

The project timeline is organized into requirement analysis, design, development, testing, and deployment phases.

6.2 Detailed Timeline Table

Month	Phase	Activities	Expected Outcome
Week 1-2	Requirement Analysis & Design	• Finalize system requirements • Prepare system architecture design • Design database schema • Design UI wireframes • Setup project repository and development environment	Clear project structure and approved design
Week 3-4	Authentication Module Development	• Implement user registration • Implement login system • Setup JWT authentication • Test authentication APIs	Secure login and registration system completed
Week 5-6	Core Module Development	• Develop expense management module • Develop habit tracking module	Fully functional expense and habit modules

		• Create RESTful APIs • Connect frontend with backend • Perform integration testing	
Week 7-8	Dashboard Development	• Implement analytics logic • Generate charts and reports • Build dashboard UI • Implement smart alert rules	Working dashboard with analytics
Week 9-10	Testing & Optimization	• Perform system testing • Fix bugs and errors • Improve performance • Validate form and error handling	Stable and optimized system
Week 11-12	Deployment & Final Preparation	• Deploy frontend • Deploy backend • Configure database hosting • Prepare documentation • Final project preparation & presentation	Live deployed system and final submission

6.3 Risk Management Plan

Possible risks and mitigation strategies are listed below:

- Delay in development: Tasks will be divided clearly between team members.
- Technical issues: Technical documentation and supervisor guidance will be followed.
- Deployment problems: Testing will be done before final deployment.

This structured timeline ensures that the project remains manageable and is completed within the academic semester.

7. System Architecture Diagram :

8. Conclusion

Smart Expense and Habit Tracker is a comprehensive web-based application designed to improve personal financial management and daily habit monitoring. The system integrates expense tracking and habit analysis into a single platform, allowing users to understand their behavior patterns through structured data and visual reports.

This project demonstrates practical implementation of full-stack web development using the MERN stack. It covers frontend development, backend API design, database management, authentication, deployment, and system testing. The project also follows proper software engineering principles including modular design, RESTful architecture, and version control.

By completing this project, the team will gain hands-on experience in real-world web application development. The system is scalable and can be extended in the future with advanced analytics, AI-based prediction models, and mobile application integration.

In conclusion, Expense AI is a practical, structured, and technically sound project that meets academic requirements while providing real-life usability and long-term development potential.